When using Hibernate, the application developer needs not to worry about the locking syntax employed by the underlying database system. To acquire an exclusive lock, the `PESSIMISTIC_WRITE` lock type must be used, and Hibernate will pick the underlying `Dialect` lock clause.

For instance, when running the following entity lock acquisition request on PostgreSQL:

```
Post post = entityManager.find(Post.class, 1L, LockModeType.PESSIMISTIC_WRITE);
```

Hibernate is going to generate the following query:

```
SELECT p.id AS id1_0_0_, p.body AS body2_0_0_, p.title AS title3_0_0_,
       p.version AS version4_0_0_
FROM   post p
WHERE  p.id = 1
FOR UPDATE
```

If the relational database offers support for acquiring shared locks explicitly, the `PESSIMISTIC_READ` lock type must be used instead. When fetching an entity directly using the `PESSIMISTIC_READ` lock type on PostgreSQL:

```
Post post = entityManager.find(Post.class, 1L, LockModeType.PESSIMISTIC_READ);
```

Hibernate is going to use the `FOR SHARE` select clause:

```
SELECT p.id AS id1_0_0_, p.body AS body2_0_0_, p.title AS title3_0_0_,
       p.version AS version4_0_0_
FROM   post p
WHERE  p.id = 1
FOR SHARE
```

If the underlying database does not support shared locks, when using the `PESSIMISTIC_READ` lock type, an exclusive lock is acquired instead. When running the previous `PESSIMISTIC_READ` direct fetching example on Oracle, Hibernate will use a `FOR UPDATE` select clause.

Although it is much more convenient to lock entities at the moment they are fetched from the database, entities can also be locked even after they are loaded in the currently running Persistence context.

For this purpose, the `EntityManager` interface defines the `lock` method which takes a managed entity and a `LockModeType`:

```
Post post = entityManager.find(Post.class, 1L);
entityManager.lock(post, LockModeType.PESSIMISTIC_WRITE);
```

When running the aforementioned example, Hibernate is going to execute the following statements:

```
SELECT p.id AS id1_0_0_, p.body AS body2_0_0_, p.title AS title3_0_0_,
       p.version AS version4_0_0_
FROM   post p
WHERE  p.id = 1

SELECT id
FROM   post
WHERE  id = 1 AND version = 0
FOR UPDATE
```

Only a managed entity can be passed to the `lock` method when using the Java Persistence API. Otherwise, an `IllegalArgumentException` is being thrown indicating that the entity is not contained within the currently running Persistence Context.

On the other hand, the Hibernate native API offers entity reattachment upon locking as demonstrated by the following example:

```
Post post = doInJPA(entityManager -> {
    return entityManager.find(Post.class, 1L);
});

doInJPA(entityManager -> {
    LOGGER.info("Lock and reattach");
    Session session = entityManager.unwrap(Session.class);
    session.buildLockRequest(
        new LockOptions(LockMode.PESSIMISTIC_WRITE))
    .lock(post);
    post.setTitle("High-Performance Hibernate");
});
```

When running the test case above, Hibernate manages to acquire an exclusive lock on the associated database record while also propagating the entity state change to the database:

```
SELECT p.id AS id1_0_0_, p.body AS body2_0_0_, p.title AS title3_0_0_,
       p.version AS version4_0_0_
FROM   post p
WHERE  p.id = 1

-- Lock and reattach
SELECT id
FROM   post
WHERE  id = 1 AND version = 0
FOR UPDATE

UPDATE post
SET body = 'Chapter 17 summary', title = High-Performance Hibernate'
WHERE id = 1 AND version = 0
```

Because the detached entity becomes managed, the entity modification triggers an update statement at flush time.

16.2.1.1 Lock scope

By default, the lock scope is bound to the entity that is being locked explicitly. However, just like other entity state transitions, the lock acquisition request can be cascaded to child associations like the `PostDetails` and `PostComment(s)` entities in the next diagram.

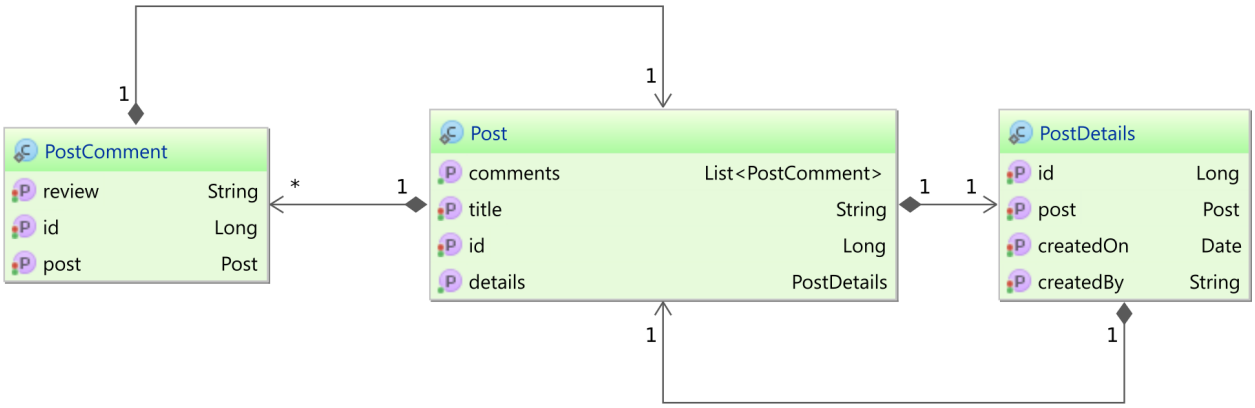

Figure 16.6: Post, PostDetails, and PostComment

The easiest way to lock a whole entity graph is to apply the `LockModeType` at the entity query level.

When executing the following entity query:

```
Post post = entityManager.createQuery(
    "select p " +
    "from Post p " +
    "join fetch p.details " +
    "join fetch p.comments " +
    "where p.id = :id", Post.class)
.setParameter("id", 1L)
.setLockMode(LockModeType.PESSIMISTIC_WRITE)
.getSingleResult();
```

Hibernate generates the following SQL query:

```
SELECT p.id AS id1_0_0_, p.body AS body2_0_0_, p.title AS title3_0_0_,
       p.version AS version4_0_0_, pd.id AS id1_2_1_,
       pd.created_by AS created_2_2_1_, pd.created_on AS created_3_2_1_,
       pd.version AS version4_2_1_, pc.id AS id1_1_2_,
       pc.post_id AS post_id4_1_2_, pc.review AS review2_1_2_,
       pc.version AS version3_1_2_, pc.post_id AS post_id4_1_0__,
       pc.id AS id1_1_0__
FROM   post p
INNER JOIN post_details pd ON p.id = pd.id
INNER JOIN post_comment pc ON p.id = pc.post_id
WHERE  p.id = 1
FOR UPDATE
```

The `FOR UPDATE` clause is applied to all records that are being selected, therefore, the whole result set is being locked. In this particular case, the lock scope depends on the query filtering criteria.

Aside from entity queries, Hibernate can also propagate a lock acquisition request from a parent entity to its children when using direct fetching. For this purpose, the child associations must be annotated with the Hibernate specific `CascadeType.Lock`[1] attribute.

`CascadeType.Lock` can also be inherited implicitly when the child association is annotated with the `CascadeType.ALL` attribute.

[1] https://docs.jboss.org/hibernate/orm/current/javadocs/org/hibernate/annotations/CascadeType.html#LOCK

To demonstrate how the lock can be cascaded, the `Post` entity is changed so that the `CascadeType.ALL` attribute is set on both `comments` and `details` child associations:

```
@OneToMany(cascade = CascadeType.ALL, mappedBy = "post", orphanRemoval = true)
private List<PostComment> comments = new ArrayList<>();

@OneToOne(cascade = CascadeType.ALL, mappedBy = "post", orphanRemoval = true,
          fetch = FetchType.LAZY, optional = false)
private PostDetails details;
```

The implicit or explicit `CascadeType.Lock` is not sufficient because the `LockRequest`[2] declares a `scope` attribute which is disabled by default. For the lock to be cascaded, the `scope` attribute must be set to true as in the following example:

```
Post post = entityManager.find(Post.class, 1L);
entityManager.unwrap(Session.class)
.buildLockRequest(
    new LockOptions(LockMode.PESSIMISTIC_WRITE))
.setScope(true)
.lock(post);
```

However, when executing the test case above, Hibernate is going to lock only the `Post` entity:

```
SELECT p.id AS id1_0_0_, p.body AS body2_0_0_, p.title AS title3_0_0_,
       p.version AS version4_0_0_
FROM   post p
WHERE  p.id = 1

SELECT id
FROM   post
WHERE  id = 1 AND version = 0
FOR UPDATE
```

For managed entities, Hibernate does not cascade the lock acquisition request even if the scope attribute is provided, therefore, the entity query alternative is preferred.

When locking a detached entity graph, Hibernate is going to reattach every entity that enabled cascade propagation while also propagating the lock request.

[2] https://docs.jboss.org/hibernate/orm/current/javadocs/org/hibernate/Session.LockRequest.html